

PROBLEM SET 3

```
-- Test pgtbltest --  
$ make qemu-gdb  
(13.1s)  
== Test   pgtbltest: ugetpid ==  
   pgtbltest: ugetpid: OK  
== Test   pgtbltest: print_kpgtbl ==  
   pgtbltest: print_kpgtbl: OK  
== Test   pgtbltest: superpg ==  
   pgtbltest: superpg: OK  
== Test answers-pgtbl.txt ==  
answers-pgtbl.txt: OK  
== Test usertests ==  
$ make qemu-gdb  
(198.3s)  
== Test time ==  
time: OK  
Score: 41/41  
rockybai@DESKTOP-9TFLS5L:~/xv6-labs-2025$
```

TASK 1 (Inspect a user-process page table)

```
init: starting sh
$ pgtbltest
print_ptbl starting
va 0x0 pte 0x21FC885B pa 0x87F22000 perm 0x5B
va 0x1000 pte 0x21FC7C5B pa 0x87F1F000 perm 0x5B
va 0x2000 pte 0x21FC7817 pa 0x87F1E000 perm 0x17
va 0x3000 pte 0x21FC7407 pa 0x87F1D000 perm 0x7
va 0x4000 pte 0x21FC70D7 pa 0x87F1C000 perm 0xD7
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFD000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFE000 pte 0x21FD08C7 pa 0x87F42000 perm 0xC7
va 0x3FFFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
print_ptbl: OK
ugetpid_test starting
usertrap(): unexpected scause 0xd pid=3
                sepc=0x79e stval=0x3fffffd000
t
```

Each Page Table Entry (PTE) maps a virtual page (VA) in the process's address space to a physical page (PA) in main memory.

The permission bits define how the CPU can access that memory (read, write, execute, user/kernel mode).

0x5B → Page is valid, readable, writable, executable, and user-accessible.

0x17 → Page is valid, readable, writable, and user-accessible, but not executable.

0x7 → Page is valid, readable, and writable, but not executable or user-accessible.

0xD7 → Page is valid, readable, writable, user-accessible, accessed, and dirty, but not executable.

0x0 → Page is unmapped; no permissions.

0xC7 → Page is valid, readable, writable, accessed, and dirty, but not executable or user-accessible.

0x4B → Page is valid, readable, executable, and accessed, but not writable or user-accessible.

The variation in permission bits shows how xv6 separates different memory regions:

Pages with 0x5B likely belong to the code/text or initialized data segment.

Pages with 0x17 are data or BSS (uninitialized data) pages — writable but not executable.

Pages with 0x7 are kernel-only or shared pages not accessible from user mode.

Pages with 0xD7 often correspond to stack or heap, accessed and dirty.

Pages with 0x0 are unmapped (holes) — no physical memory assigned.

The last two pages (0xC7 and 0x4B) are kernel mappings for trapframes or trampoline code used in system calls and interrupts.

The use trap at the end indicates a store page fault (scause = 0xD), which occurs when a user process attempts to write to a virtual address that is either unmapped or not writable.

The faulting address (stval = 0x3FFFFFFD000) lies in the upper end of the user virtual address space, and the page table output confirms that this virtual page has no valid mapping (PTE = 0x0).

Task 2 (Speed up system calls)

Other system calls that only return simple read-only information can also be sped up using a shared page between the kernel and user space. For example:

getppid() – It just returns the parent process ID, which doesn't change often. Storing it in the shared page would let the process read it directly instead of calling into the kernel.

uptime() – This could also be optimized by keeping the current tick count in the shared page. The kernel can update it periodically, and user programs can read it directly without a trap.

getuid() / **geteuid()** – These values are static for most processes, so they can easily be kept in the same shared page for fast access.

Basically, any system call that only reads data and doesn't modify kernel state can benefit from this method. It removes the need for switching between user and kernel mode, which saves time and makes frequent calls faster.

Task 3(Print a page table)

Each **leaf page** in the vmprint output represents the final level of translation in the page table hierarchy—i.e., entries that directly map virtual addresses to physical memory. Logically, these leaf pages contain the actual code, data, stack, or other user-space segments of the process. For instance, pages with virtual addresses near 0x0 typically contain the program's **text (code) and data segments**, while higher addresses such as 0x3FFFFFFE000 or 0x3FFFFFFF000 usually correspond to the **trapframe** or **kernel-mapped regions** like the trampoline. The permission bits (e.g., 0x5B, 0x17, 0xC7, 0x4B) describe how each page can be accessed — whether it's readable, writable, executable, and whether user mode has access. These match the meanings used in the earlier print_pgtbl() output: for example, a page with permission 0x5B is user-accessible, readable, writable, and executable, whereas 0x17 lacks the user bit and may be kernel-only. Thus, the leaf entries in vmprint directly correspond to the same final-level PTEs shown earlier in print_pgtbl(), but now visualized in their full hierarchical structure, showing how the virtual memory space of the process is organized and linked to physical pages.